

Verification Final Project

UVM Project - SPI Slave with Single Port RAM

By:

NAME	Group
Ahmed Ibrahim Sabry Ismail	2
Ezelden Essam Negm	1
Mohamed Haitham Mokhtar	1

Sent by: Ahmed Ibrahim Sabry Ismail

PART 1: SPI slave

Verification plan:

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functional Check
SPI_1	When reset (rst_n) is asserted low, all DUT outputs (MISO, rx_data, rx_valid) and internal states must be cleared.	Apply reset at the start of simulation and occasionally during operation (5% of total time).	-	Concurrent assertion p1 checks that when reset is asserted, all outputs are cleared.
SPI_2	After any valid command (000, 001, 110, 111), rx_valid must assert exactly after 10 clock cycles and SS_n should eventually rise to close communication.I.	Randomize mosi_bits[10:8] to generate all command types with equal probability. Repeat commands throughout the test.	Coverpoint on seq_item.mosi_bits[10:8] to ensure all valid command types appear.	Assertion p2 ensures rx_valid asserts exactly 10 cycles after command start, and SS_n rises eventually.
SPI_3	rx_valid high must be followed by SS_n rising edge to indicate proper frame completion.	Apply sequences with variable frame lengths and delays to verify protocol completion.	Cross coverage between rx_valid and SS_n transitions.	Assertion p3 checks that \$rose(rx_valid) eventually leads to \$rose(SS_n).
SPI_4	State machine transitions must follow: IDLE → CHK_CMD → WRITE/READ_ADD/READ_DATA, and return to IDLE after SS_n high.	Drive multiple command types and random SS_n toggling.	Covergroup on FSM states cs[2:0] to confirm all valid transitions.	Assertions p4-p7 check legal state transitions between FSM states.
SPI_5	During a write sequence, data must be correctly captured in rx_data and validated with rx_valid_ref.	Send continuous write frames (000, 001) with random data.	Cross coverage on ss_n_cp and mosi_cp to verify write/read transitions.	Scoreboard compares rx_data with rx_data_ref each frame.
SPI_6	MISO output must match reference model output MISO_ref for all valid transactions.	Apply both write and read commands repeatedly.	Functional coverage on MISO transitions can be optional	Scoreboard performs bit-by-bit comparison of MISO and MISO_ref.

Functional Coverage report:

Coverage Report by instance with details

```
=====
=====
=== Instance: /top/DUT1/alu_assertion_inst
=== Design Unit: work.spi_slave_assertions
=====
=====
```

Directive Coverage:

Directives	3	3	0	100.00%
------------	---	---	---	---------

DIRECTIVE COVERAGE:

Name	Design Unit	Design UnitType	Lang	File(Line)	Hits	Status
------	-------------	-----------------	------	------------	------	--------

/top/DUT1/alu_assertion_inst/cp1 SPI_assertion.sv(33)			spi_slave_assertions	Verilog	SVA	1845 Covered
/top/DUT1/alu_assertion_inst/cp2 SPI_assertion.sv(35)			spi_slave_assertions	Verilog	SVA	1316 Covered
/top/DUT1/alu_assertion_inst/cp3 SPI_assertion.sv(37)			spi_slave_assertions	Verilog	SVA	1595 Covered

=====
 Instance: /top/DUT1
 Design Unit: work.SLAVE
 =====

Directive Coverage:
 Directives 5 5 0 100.00%

DIRECTIVE COVERAGE:

Name	Design Unit	Design UnitType	Lang	File(Line)	Hits	Status
------	-------------	-----------------	------	------------	------	--------

/top/DUT1/cp4	SLAVE		Verilog	SVA SPI_slave.sv(154)	1896 Covered	
/top/DUT1/cp5	SLAVE		Verilog	SVA SPI_slave.sv(156)	1896 Covered	
/top/DUT1/cp6	SLAVE		Verilog	SVA SPI_slave.sv(158)	948 Covered	
/top/DUT1/cp7	SLAVE		Verilog	SVA SPI_slave.sv(160)	367 Covered	
/top/DUT1/cp8	SLAVE		Verilog	SVA SPI_slave.sv(162)	581 Covered	

=====
 Instance: /SPI_coverage1_pkg
 Design Unit: work.SPI_coverage1_pkg

=====

Covergroup Coverage:

Covergroups	1	na	na	100.00%
Coverpoints/Crosses	4	na	na	na
Covergroup Bins	20	20	0	100.00%

Covergroup	Metric	Goal	Bins	Status
TYPE /SPI_coverage1_pkg/SPI_coverage1/g1		100.00%	100	-
Covered				
covered/total bins:	20	20	-	
missing/total bins:	0	20	-	
% Hit:	100.00%	100	-	
Coverpoint rx_data_cp	100.00%	100	-	Covered
covered/total bins:	7	7	-	
missing/total bins:	0	7	-	
% Hit:	100.00%	100	-	
Coverpoint ss_n_cp	100.00%	100	-	Covered
covered/total bins:	4	4	-	
missing/total bins:	0	4	-	
% Hit:	100.00%	100	-	
Coverpoint mosi_cp	100.00%	100	-	Covered
covered/total bins:	6	6	-	
missing/total bins:	0	6	-	
% Hit:	100.00%	100	-	
Cross cross1	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	
% Hit:	100.00%	100	-	
Covergroup instance \SPI_coverage1_pkg::SPI_coverage1::g1	100.00%	100	-	Covered
covered/total bins:	20	20	-	
missing/total bins:	0	20	-	
% Hit:	100.00%	100	-	
Coverpoint rx_data_cp	100.00%	100	-	Covered
covered/total bins:	7	7	-	
missing/total bins:	0	7	-	
% Hit:	100.00%	100	-	
bin all_values[0]	10721	1	-	Covered
bin all_values[1]	8638	1	-	Covered
bin all_values[2]	10132	1	-	Covered

bin all_values[3]	5664	1	-	Covered
bin transitions[0=>1]	298	1	-	Covered
bin transitions[2=>3]	120	1	-	Covered
bin transitions[3=>0]	13	1	-	Covered
Coverpoint ss_n_cp	100.00%	100	-	Covered
covered/total bins:	4	4	-	
missing/total bins:	0	4	-	
% Hit:	100.00%	100	-	
bin values1[0]	31145	1	-	Covered
bin values1[1]	4010	1	-	Covered
bin normal_txn	1496	1	-	Covered
bin read_txn	508	1	-	Covered
Coverpoint mosi_cp	100.00%	100	-	Covered
covered/total bins:	6	6	-	
missing/total bins:	0	6	-	
% Hit:	100.00%	100	-	
bin values2[0]	7560	1	-	Covered
bin values2[1]	7575	1	-	Covered
bin values2[6]	7320	1	-	Covered
bin values2[7]	12700	1	-	Covered
bin write_sequence	139	1	-	Covered
bin read_sequence	145	1	-	Covered
Cross cross1	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	
% Hit:	100.00%	100	-	
Auto, Default and User Defined Bins:				
bin cross_bins	31145	1	-	Covered
bin <values1[0],read_sequence>		145	1	- Covered
bin <values1[0],write_sequence>		139	1	- Covered
Illegal and Ignore Bins:				
ignore_bin ignored_bins	4010		-	Occurred

COVERGROUP COVERAGE:

Covergroup	Metric	Goal	Bins	Status
TYPE /SPI_coverage1_pkg/SPI_coverage1/g1		100.00%	100	-
Covered				
covered/total bins:	20	20	-	
missing/total bins:	0	20	-	
% Hit:	100.00%	100	-	
Coverpoint rx_data_cp	100.00%	100	-	Covered

covered/total bins:	7	7	-	
missing/total bins:	0	7	-	
% Hit:	100.00%	100	-	
Coverpoint ss_n_cp	100.00%	100	-	Covered
covered/total bins:	4	4	-	
missing/total bins:	0	4	-	
% Hit:	100.00%	100	-	
Coverpoint mosi_cp	100.00%	100	-	Covered
covered/total bins:	6	6	-	
missing/total bins:	0	6	-	
% Hit:	100.00%	100	-	
Cross cross1	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	
% Hit:	100.00%	100	-	
Covergroup instance VSPI_coverage1_pkg::SPI_coverage1::g1	100.00%	100	-	Covered
covered/total bins:	20	20	-	
missing/total bins:	0	20	-	
% Hit:	100.00%	100	-	
Coverpoint rx_data_cp	100.00%	100	-	Covered
covered/total bins:	7	7	-	
missing/total bins:	0	7	-	
% Hit:	100.00%	100	-	
bin all_values[0]	10721	1	-	Covered
bin all_values[1]	8638	1	-	Covered
bin all_values[2]	10132	1	-	Covered
bin all_values[3]	5664	1	-	Covered
bin transitions[0=>1]	298	1	-	Covered
bin transitions[2=>3]	120	1	-	Covered
bin transitions[3=>0]	13	1	-	Covered
Coverpoint ss_n_cp	100.00%	100	-	Covered
covered/total bins:	4	4	-	
missing/total bins:	0	4	-	
% Hit:	100.00%	100	-	
bin values1[0]	31145	1	-	Covered
bin values1[1]	4010	1	-	Covered
bin normal_txn	1496	1	-	Covered
bin read_txn	508	1	-	Covered
Coverpoint mosi_cp	100.00%	100	-	Covered
covered/total bins:	6	6	-	
missing/total bins:	0	6	-	
% Hit:	100.00%	100	-	
bin values2[0]	7560	1	-	Covered

bin values2[1]	7575	1	-	Covered
bin values2[6]	7320	1	-	Covered
bin values2[7]	12700	1	-	Covered
bin write_sequence	139	1	-	Covered
bin read_sequence	145	1	-	Covered
Cross cross1	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	
% Hit:	100.00%	100	-	
Auto, Default and User Defined Bins:				
bin cross_bins	31145	1	-	Covered
bin <values1[0],read_sequence>		145	1	- Covered
bin <values1[0],write_sequence>		139	1	- Covered
Illegal and Ignore Bins:				
ignore_bin ignored_bins	4010		-	Occurred

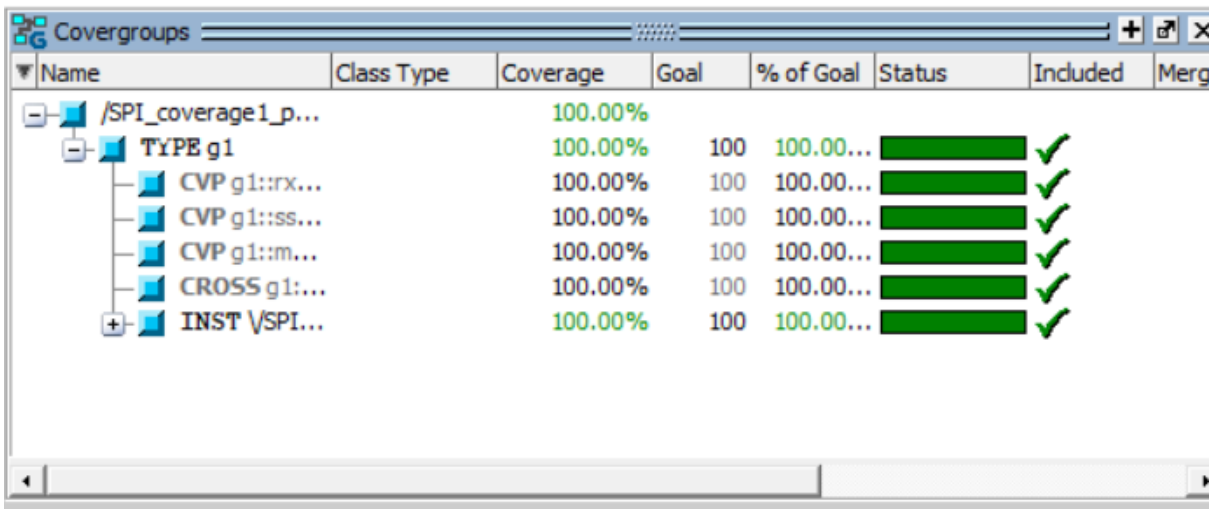
TOTAL COVERGROUP COVERAGE: 100.00% COVERGROUP TYPES: 1

DIRECTIVE COVERAGE:

Name	Design Unit	Design UnitType	Lang	File(Line)	Hits	Status
/top/DUT1/cp4	SLAVE	Verilog	SVA	SPI_slave.sv(154)	1896	Covered
/top/DUT1/cp5	SLAVE	Verilog	SVA	SPI_slave.sv(156)	1896	Covered
/top/DUT1/cp6	SLAVE	Verilog	SVA	SPI_slave.sv(158)	948	Covered
/top/DUT1/cp7	SLAVE	Verilog	SVA	SPI_slave.sv(160)	367	Covered
/top/DUT1/cp8	SLAVE	Verilog	SVA	SPI_slave.sv(162)	581	Covered
/top/DUT1/alu_assertion_inst/cp1 SPI_assertion.sv(33)		spi_slave_assertions	Verilog	SVA	1845	Covered
/top/DUT1/alu_assertion_inst/cp2 SPI_assertion.sv(35)		spi_slave_assertions	Verilog	SVA	1316	Covered
/top/DUT1/alu_assertion_inst/cp3 SPI_assertion.sv(37)		spi_slave_assertions	Verilog	SVA	1595	Covered

TOTAL DIRECTIVE COVERAGE: 100.00% COVERS: 8

Total Coverage By Instance (filtered view): 100.00%



The screenshot shows the Covergroups tool interface. On the left, a tree view displays the hierarchy: /SPI_coverage1_p... expanded to show TYPE g1, which contains CVP g1::rx..., CVP g1::ss..., CVP g1::m..., CROSS g1::..., and INST \SPI... All items are expanded. On the right, a table lists the coverage data for these groups.

Name	Class Type	Coverage	Goal	% of Goal	Status	Included	Merg
/SPI_coverage1_p...		100.00%					
TYPE g1		100.00%	100	100.00...		✓	
CVP g1::rx...		100.00%	100	100.00...		✓	
CVP g1::ss...		100.00%	100	100.00...		✓	
CVP g1::m...		100.00%	100	100.00...		✓	
CROSS g1::...		100.00%	100	100.00...		✓	
INST \SPI...		100.00%	100	100.00...		✓	

Code Coverage report and assertion:

Coverage Report by instance with details

```
=====
=====
=== Instance: /top#DUT1 /alu_assertion_inst
=== Design Unit: work.spi_slave_assertions
=====
=====
```

Assertion Coverage:

Assertions	6	6	0	100.00%
------------	---	---	---	---------

Name	File(Line)	Failure Count	Pass Count
------	------------	---------------	------------

/top#DUT1 /alu_assertion_inst/ap1			
SPI_assertion.sv(55)		0	1
/top#DUT1 /alu_assertion_inst/ap2			
SPI_assertion.sv(57)		0	1
/top#DUT1 /alu_assertion_inst/ap3			

SPI_assertion.sv(59)	0	1
^top#DUT1 /alu_assertion_inst/ap4		
SPI_assertion.sv(61)	0	1
^top#DUT1 /alu_assertion_inst/ap5		
SPI_assertion.sv(63)	0	1
^top#DUT1 /alu_assertion_inst/ap6		
SPI_assertion.sv(65)	0	1

Directive Coverage:

Directives	6	6	0	100.00%
------------	---	---	---	---------

DIRECTIVE COVERAGE:

Name	Design Unit	Design UnitType	Lang	File(Line)	Hits	Status
^top#DUT1 /alu_assertion_inst/cp1 SPI_assertion.sv(56)			spi_slave_assertions	Verilog	SVA	
					1845	Covered
^top#DUT1 /alu_assertion_inst/cp2 SPI_assertion.sv(58)			spi_slave_assertions	Verilog	SVA	
					478	Covered
^top#DUT1 /alu_assertion_inst/cp3 SPI_assertion.sv(60)			spi_slave_assertions	Verilog	SVA	
					470	Covered
^top#DUT1 /alu_assertion_inst/cp4 SPI_assertion.sv(62)			spi_slave_assertions	Verilog	SVA	
					461	Covered
^top#DUT1 /alu_assertion_inst/cp5 SPI_assertion.sv(64)			spi_slave_assertions	Verilog	SVA	
					186	Covered
^top#DUT1 /alu_assertion_inst/cp6 SPI_assertion.sv(66)			spi_slave_assertions	Verilog	SVA	
					1595	Covered

Toggle Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	-----	-----	-----
Toggles	38	38	0	100.00%

=====Toggle
Details=====

Toggle Coverage for instance ^top#DUT1 /alu_assertion_inst --

Node	1H->0L	0L->1H	"Coverage"
MISO	1	1	100.00
MOSI	1	1	100.00
SS_n	1	1	100.00
clk	1	1	100.00
cs[2-0]	1	1	100.00
rst_n	1	1	100.00
rx_data[9-0]	1	1	100.00
rx_valid	1	1	100.00

Total Node Count = 19
 Toggled Node Count = 19
 Untoggled Node Count = 0

Toggle Coverage = 100.00% (38 of 38 bins)

```
=====
=====
=== Instance: /top#DUT1
=== Design Unit: work.SLAVE
=====
=====
```

Assertion Coverage:

Assertions	5	5	0	100.00%
------------	---	---	---	---------

Name	File(Line)	Failure Count	Pass Count
------	------------	---------------	------------

/top#DUT1 /ap4	SPI_slave.sv(153)	0	1
/top#DUT1 /ap5	SPI_slave.sv(155)	0	1
/top#DUT1 /ap6	SPI_slave.sv(157)	0	1
/top#DUT1 /ap7	SPI_slave.sv(159)	0	1
/top#DUT1 /ap8	SPI_slave.sv(161)	0	1

Branch Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
Branches	33	33	0	100.00%

```
=====Branch
Details=====
```

Branch Coverage for instance /top#DUT1

Line	Item	Count	Source
----	----	-----	-----
File SPI_slave.sv			
-----IF Branch-----			
20		9699	Count coming in to IF
20	1	219	if (~rst_n) begin
23	1	9480	else begin
Branch totals: 2 hits of 2 branches = 100.00%			
-----CASE Branch-----			
29		20373	Count coming in to CASE
30	1	4592	IDLE : begin
36	1	2835	CHK_CMD : begin
50	1	6100	WRITE : begin
56	1	2741	READ_ADD : begin
62	1	4104	READ_DATA : begin
		1	All False Count
Branch totals: 6 hits of 6 branches = 100.00%			
-----IF Branch-----			
31		4592	Count coming in to IF
31	1	2005	if (SS_n)
33	1	2587	else
Branch totals: 2 hits of 2 branches = 100.00%			
-----IF Branch-----			
37		2835	Count coming in to IF
39	1	2835	else begin
Branch totals: 1 hit of 1 branch = 100.00%			
-----IF Branch-----			
40		2835	Count coming in to IF
40	1	1430	if (~MOSI)
42	1	1405	else begin
Branch totals: 2 hits of 2 branches = 100.00%			
-----IF Branch-----			
43		1405	Count coming in to IF
43	1	849	if (received_address)
45	1	556	else
Branch totals: 2 hits of 2 branches = 100.00%			
-----IF Branch-----			

51		6100	Count coming in to IF
51	1	948	if (SS_n)
53	1	5152	else

Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----

57		2741	Count coming in to IF
57	1	367	if (SS_n)
59	1	2374	else

Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----

63		4104	Count coming in to IF
63	1	581	if (SS_n)
65	1	3523	else

Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----

72		29122	Count coming in to IF
72	1	683	if (~rst_n) begin
79	1	28439	else begin

Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----

88		11376	Count coming in to IF
88	1	9480	if (counter > 0) begin
92	1	1896	else begin

Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----

97		4404	Count coming in to IF
97	1	3670	if (counter > 0) begin
101	1	734	else begin

Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----

107		6972	Count coming in to IF
107	1	3612	if (tx_valid) begin
117	1	3360	else begin

Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----

109		3612	Count coming in to IF
109	1	3010	if (counter > 0) begin

```

113      1      602      else begin
Branch totals: 2 hits of 2 branches = 100.00%

```

```

-----IF Branch-----
118      3360  Count coming in to IF
118      1      3080      if (counter > 0) begin
122      1      280      else begin
Branch totals: 2 hits of 2 branches = 100.00%

```

```

Condition Coverage:
Enabled Coverage      Bins  Covered  Misses  Coverage
-----
Conditions            4      4      0  100.00%

```

```

=====Condition
Details=====

```

Condition Coverage for instance /atop#DUT1 --

```

File SPI_slave.sv
-----Focused Condition View-----
Line   88 Item   1 (counter > 0)
Condition totals: 1 of 1 input term covered = 100.00%

```

Input Term	Covered	Reason for no coverage	Hint
(counter > 0)	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
Row 1:	1	(counter > 0)_0	-
Row 2:	1	(counter > 0)_1	-

```

-----Focused Condition View-----
Line   97 Item   1 (counter > 0)
Condition totals: 1 of 1 input term covered = 100.00%

```

Input Term	Covered	Reason for no coverage	Hint
(counter > 0)	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
-------	------	------------	--------------------------

Row 1:	1 (counter > 0)_0	-
Row 2:	1 (counter > 0)_1	-

-----Focused Condition View-----

Line 109 Item 1 (counter > 0)

Condition totals: 1 of 1 input term covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
(counter > 0)	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
Row 1:	1	(counter > 0)_0	-
Row 2:	1	(counter > 0)_1	-

-----Focused Condition View-----

Line 118 Item 1 (counter > 0)

Condition totals: 1 of 1 input term covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
(counter > 0)	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
Row 1:	1	(counter > 0)_0	-
Row 2:	1	(counter > 0)_1	-

Directive Coverage:

Directives	5	5	0	100.00%
------------	---	---	---	---------

DIRECTIVE COVERAGE:

Name	Design Unit	Design UnitType	Lang	File(Line)	Hits	Status

^top#DUT1 /cp4	SLAVE	Verilog	SVA	SPI_slave.sv(154)		
					1896	Covered
^top#DUT1 /cp5	SLAVE	Verilog	SVA	SPI_slave.sv(156)		
					1896	Covered
^top#DUT1 /cp6	SLAVE	Verilog	SVA	SPI_slave.sv(158)		

```

                                     948 Covered
^top#DUT1 /cp7                     SLAVE Verilog SVA SPI_slave.sv(160)
                                     367 Covered
^top#DUT1 /cp8                     SLAVE Verilog SVA SPI_slave.sv(162)
                                     581 Covered

```

FSM Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	-----	-----	
FSM States	5	5	0	100.00%
FSM Transitions	8	7	1	87.50%

=====FSM
Details=====

FSM Coverage for instance ^top#DUT1 --

FSM_ID: cs

Current State Object : cs

State Value MapInfo :

Line	State Name	Value
30	IDLE	0
36	CHK_CMD	2
56	READ_ADD	3
62	READ_DATA	4
50	WRITE	1

Covered States :

State	Hit_count
IDLE	4011
CHK_CMD	1896
READ_ADD	734
READ_DATA	1162
WRITE	1896

Covered Transitions :

Line	Trans_ID	Hit_count	Transition
34	0	1896	IDLE -> CHK_CMD
46	1	367	CHK_CMD -> READ_ADD
44	2	581	CHK_CMD -> READ_DATA

41	3	948	CHK_CMD -> WRITE
58	5	367	READ_ADD -> IDLE
64	6	581	READ_DATA -> IDLE
52	7	948	WRITE -> IDLE

Uncovered Transitions :

```

-----
Line      Trans_ID      Transition
-----
38         4      CHK_CMD -> IDLE

```

Summary	Bins	Hits	Misses	Coverage
-----	----	-----	-----	
FSM States	5	5	0	100.00%
FSM Transitions	8	7	1	87.50%

Statement Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	-----	-----	
Statements	38	38	0	100.00%

=====Statement
Details=====

Statement Coverage for instance /top#DUT1 --

Line	Item	Count	Source
----	----	-----	-----

File SPI_slave.sv

```

1      module SLAVE
(MOSI,MISO,SS_n,clk,rst_n,rx_data,rx_valid,tx_data,tx_valid);
2
3      localparam IDLE    = 3'b000;
4      localparam WRITE   = 3'b001;
5      localparam CHK_CMD = 3'b010;
6      localparam READ_ADD = 3'b011;
7      localparam READ_DATA = 3'b100;
8
9      input      MOSI, clk, rst_n, SS_n, tx_valid;
10     input  [7:0] tx_data;
11     output reg [9:0] rx_data;
12     output reg  rx_valid, MISO;
13
14     reg [3:0] counter;
15     reg  received_address;

```



```

16
17         reg [2:0] cs, ns;
18
19         1          9699    always @(posedge clk) begin
20             if (~rst_n) begin
21                 1          219        cs <= IDLE;
22             end
23             else begin
24                 1          9480       cs <= ns;
25             end
26         end
27
28         1          20373   always @(*) begin
29             case (cs)
30                 IDLE : begin
31                     if (SS_n)
32                         1          2005       ns = IDLE;
33                     else
34                         1          2587       ns = CHK_CMD;
35                 end
36                 CHK_CMD : begin
37                     if (SS_n)
38                         ns = IDLE;
39                     else begin
40                         if (~MOSI)
41                             1          1430     ns = WRITE;
42                         else begin
43                             if (received_address)
44                                 1          849       ns = READ_DATA;
45                             else
46                                 1          556       ns = READ_ADD;
47                         end
48                     end
49                 end
50                 WRITE : begin
51                     if (SS_n)
52                         1          948       ns = IDLE;
53                     else
54                         1          5152      ns = WRITE;
55                 end
56                 READ_ADD : begin
57                     if (SS_n)
58                         1          367       ns = IDLE;
59                     else

```

60	1	2374	ns = READ_ADD;
61			end
62			READ_DATA : begin
63			if (SS_n)
64	1	581	ns = IDLE;
65			else
66	1	3523	ns = READ_DATA;
67			end
68			endcase
69			end
70			
71	1	29122	always @(posedge clk) begin
72			if (~rst_n) begin
73	1	683	rx_data <= 0;
74	1	683	rx_valid <= 0;
75	1	683	received_address <= 0;
76	1	683	MISO <= 0;
77	1	683	counter<=0;
78			end
79			else begin
80			case (cs)
81			IDLE : begin
82	1	3791	rx_valid <= 0;
83			end
84			CHK_CMD : begin
85	1	1896	counter <= 10;
86			end
87			WRITE : begin
88			if (counter > 0) begin
89	1	9480	rx_data[counter-1] <= MOSI;
90	1	9480	counter <= counter - 1;
91			end
92			else begin
93	1	1896	rx_valid <= 1;
94			end
95			end
96			READ_ADD : begin
97			if (counter > 0) begin
98	1	3670	rx_data[counter-1] <= MOSI;
99	1	3670	counter <= counter - 1;
100			end
101			else begin
102	1	734	rx_valid <= 1;
103	1	734	received_address <= 1;

```

104                                     end
105                                     end
106                                     READ_DATA : begin
107                                         if (tx_valid) begin
108                                             1          3612          rx_valid <= 0;
109                                         if (counter > 0) begin
110                                             1          3010          MISO <= tx_data[counter-1];
111                                             1          3010          counter <= counter - 1;
112                                         end
113                                         else begin
114                                             1          602          received_address <= 0;
115                                         end
116                                     end
117                                     else begin
118                                         if (counter > 0) begin
119                                             1          3080          rx_data[counter-1] <= MOSI;
120                                             1          3080          counter <= counter - 1;
121                                         end
122                                         else begin
123                                             1          280          rx_valid <= 1;
124                                             1          280          counter <= 8;

```

Toggle Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	-----	-----	
Toggles	72	72	0	100.00%

=====Toggle
Details=====

Toggle Coverage for instance /top#DUT1 --

Node	1H->0L	0L->1H	"Coverage"
MISO	1	1	100.00
MOSI	1	1	100.00
SS_n	1	1	100.00
clk	1	1	100.00
counter[3-0]	1	1	100.00
cs[2-0]	1	1	100.00
ns[2-0]	1	1	100.00
received_address		1 1	100.00
rst_n	1	1	100.00
rx_data[9-0]	1	1	100.00

rx_valid	1	1	100.00
tx_data[0-7]	1	1	100.00
tx_valid	1	1	100.00

Total Node Count = 36
 Toggled Node Count = 36
 Untoggled Node Count = 0

Toggle Coverage = 100.00% (72 of 72 bins)

DIRECTIVE COVERAGE:

Name	Design Unit	Design UnitType	Lang	File(Line)	Hits	Status
^top#DUT1 /cp4	SLAVE	Verilog	SVA	SPI_slave.sv(154)	1896	Covered
^top#DUT1 /cp5	SLAVE	Verilog	SVA	SPI_slave.sv(156)	1896	Covered
^top#DUT1 /cp6	SLAVE	Verilog	SVA	SPI_slave.sv(158)	948	Covered
^top#DUT1 /cp7	SLAVE	Verilog	SVA	SPI_slave.sv(160)	367	Covered
^top#DUT1 /cp8	SLAVE	Verilog	SVA	SPI_slave.sv(162)	581	Covered
^top#DUT1 /alu_assertion_inst/cp1 SPI_assertion.sv(56)			spi_slave_assertions	Verilog	SVA	1845 Covered
^top#DUT1 /alu_assertion_inst/cp2 SPI_assertion.sv(58)			spi_slave_assertions	Verilog	SVA	478 Covered
^top#DUT1 /alu_assertion_inst/cp3 SPI_assertion.sv(60)			spi_slave_assertions	Verilog	SVA	470 Covered
^top#DUT1 /alu_assertion_inst/cp4 SPI_assertion.sv(62)			spi_slave_assertions	Verilog	SVA	461 Covered
^top#DUT1 /alu_assertion_inst/cp5 SPI_assertion.sv(64)			spi_slave_assertions	Verilog	SVA	186 Covered
^top#DUT1 /alu_assertion_inst/cp6 SPI_assertion.sv(66)			spi_slave_assertions	Verilog	SVA	1595 Covered

TOTAL DIRECTIVE COVERAGE: 100.00% COVERS: 11

ASSERTION RESULTS:

Name	File(Line)	Failure Count	Pass Count
^top#DUT1 /ap4	SPI_slave.sv(153)	0	1
^top#DUT1 /ap5	SPI_slave.sv(155)	0	1
^top#DUT1 /ap6	SPI_slave.sv(157)	0	1
^top#DUT1 /ap7	SPI_slave.sv(159)	0	1
^top#DUT1 /ap8	SPI_slave.sv(161)	0	1
^top#DUT1 /alu_assertion_inst/ap1	SPI_assertion.sv(55)	0	1
^top#DUT1 /alu_assertion_inst/ap2	SPI_assertion.sv(57)	0	1
^top#DUT1 /alu_assertion_inst/ap3	SPI_assertion.sv(59)	0	1
^top#DUT1 /alu_assertion_inst/ap4	SPI_assertion.sv(61)	0	1
^top#DUT1 /alu_assertion_inst/ap5	SPI_assertion.sv(63)	0	1
^top#DUT1 /alu_assertion_inst/ap6	SPI_assertion.sv(65)	0	1

Total Coverage By Instance (filtered view): 98.43%

Name	Language	Enabled	Log	Count	AtLeast	Limit	Weight	Cmplt %	Cmplt graph	Included	Memory	Peak Memory	Peak Memory Time	Cumulative Threads
▲ /top/DUT1/cp5	SVA	✓	Off	1896	1	Unli...	1	100%	████████	✓	0	0	0 ns	0
▲ /top/DUT1/cp4	SVA	✓	Off	1896	1	Unli...	1	100%	████████	✓	0	0	0 ns	0
▲ /top/DUT1/alu_ass...	SVA	✓	Off	1845	1	Unli...	1	100%	████████	✓	0	0	0 ns	0
▲ /top/DUT1/alu_ass...	SVA	✓	Off	1595	1	Unli...	1	100%	████████	✓	0	0	0 ns	0
▲ /top/DUT1/cp6	SVA	✓	Off	948	1	Unli...	1	100%	████████	✓	0	0	0 ns	0
▲ /top/DUT1/cp8	SVA	✓	Off	581	1	Unli...	1	100%	████████	✓	0	0	0 ns	0
▲ /top/DUT1/alu_ass...	SVA	✓	Off	478	1	Unli...	1	100%	████████	✓	0	0	0 ns	0
▲ /top/DUT1/alu_ass...	SVA	✓	Off	470	1	Unli...	1	100%	████████	✓	0	0	0 ns	0
▲ /top/DUT1/alu_ass...	SVA	✓	Off	461	1	Unli...	1	100%	████████	✓	0	0	0 ns	0
▲ /top/DUT1/cp7	SVA	✓	Off	367	1	Unli...	1	100%	████████	✓	0	0	0 ns	0
▲ /top/DUT1/alu_ass...	SVA	✓	Off	186	1	Unli...	1	100%	████████	✓	0	0	0 ns	0

▲ /SPI_main_sequen...	Immediate	SVA	on	0	1	-	-	-	-	-	off	assert (randomize(...))	✓
▲ /top/DUT1/ap4	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge clk) disable iff (...))	✓
▲ /top/DUT1/ap5	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge clk) disable iff (...))	✓
▲ /top/DUT1/ap6	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge clk) disable iff (...))	✓
▲ /top/DUT1/ap7	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge clk) disable iff (...))	✓
▲ /top/DUT1/ap8	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge clk) disable iff (...))	✓
▲ /top/DUT1/alu_ass...	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge clk) (~rst_n))=...	✓
▲ /top/DUT1/alu_ass...	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge clk) disable iff (...))	✓
▲ /top/DUT1/alu_ass...	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge clk) disable iff (...))	✓
▲ /top/DUT1/alu_ass...	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge clk) disable iff (...))	✓
▲ /top/DUT1/alu_ass...	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge clk) disable iff (...))	✓
▲ /top/DUT1/alu_ass...	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge clk) disable iff (...))	✓

Feature	Assertion
Reset ensures tx_valid and dout are low	@(posedge clk) !rst_n => (MISO == 1'b0 && rx_valid == 1'b0 && rx_data == '0)
Detects a falling edge on SS_n followed by three consecutive cycles where MOSI remains 0	\$fell(SS_n) ##1 MOSI == 0 ##1 MOSI == 0 ##1 MOSI == 0;
When s2 becomes true, after 10 cycles, rx_valid must go high, and immediately after that, SS_n must go high for one cycle	@(posedge clk) disable iff (!rst_n)s2 -> ##10 (rx_valid == 1'b1) -> (SS_n == 1'b1) [->1];
Detects a falling edge on SS_n followed by two consecutive cycles where MOSI is 0, then MOSI becomes 1 on the next cycle	\$fell(SS_n) ##1 MOSI == 0 ##1 MOSI == 0 ##1 MOSI == 1;
When s3 becomes true, after 10 cycles, rx_valid must go high, and immediately after that, SS_n must go high for one cycle	s3 -> ##10 (rx_valid == 1'b1) -> (SS_n == 1'b1) [->1];
Detects a falling edge on SS_n followed by two consecutive cycles where MOSI is 1, then MOSI becomes 0 on the next cycle.	\$fell(SS_n) ##1 MOSI == 1 ##1 MOSI == 1 ##1 MOSI == 0;
When s4 becomes true, after 10 cycles, rx_valid must go high, and immediately after that, SS_n must go high for one cycle	'@(posedge clk) disable iff (!rst_n)s4 -> ##10 (rx_valid == 1'b1) -> (SS_n == 1'b1) [->1];
Detects a falling edge on SS_n followed by three consecutive cycles where MOSI remains 1.	\$fell(SS_n) ##1 MOSI == 1 ##1 MOSI == 1 ##1 MOSI == 1;
When s5 becomes true, after 10 cycles, rx_valid must go high, and immediately after that, SS_n must go high for one cycle.	'@(posedge clk) disable iff (!rst_n)s5 -> ##10 (rx_valid == 1'b1) -> (SS_n == 1'b1) [->1];
When rx_valid rises, it must eventually be followed by a rising edge of SS_n.	'@(posedge clk) disable iff (!rst_n)\$rose(rx_valid) -> s_eventually \$rose(SS_n)

- 1) I made exclusion on branch case (cs).
- 2) I made exclusion on statement ns = IDLE; line 38.

Bugs:

- 1) make the next state READ_DATA when the received_address is high.

Before:

```

ns = IDLE;
else begin
    if (received_address)
        ns = READ_ADD;
    else
        ns = READ_DATA;
end

```

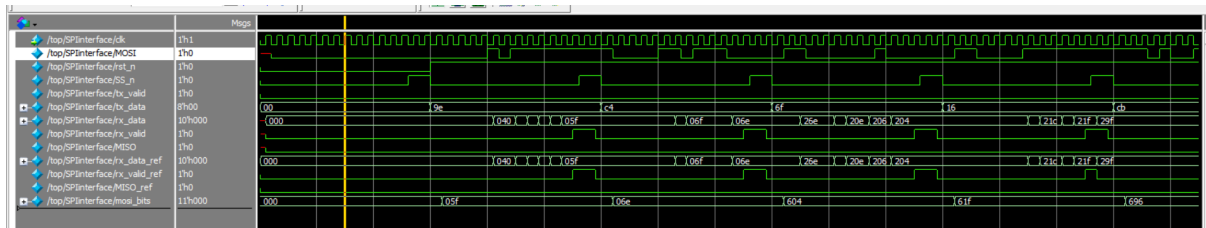
After:

```

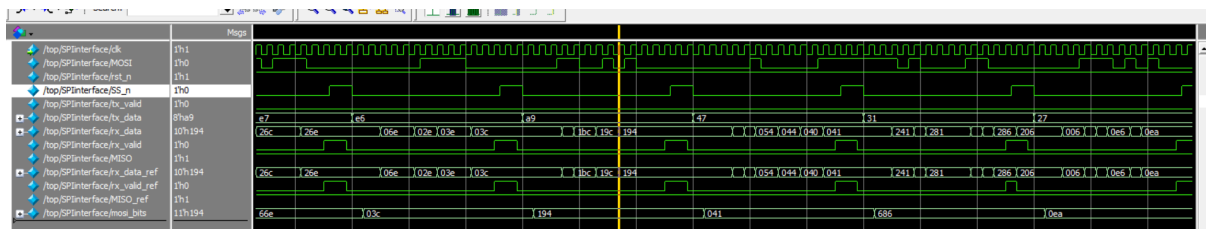
else begin
    if (received_address)
        ns = READ_DATA;
    else
        ns = READ_ADD;
end

```

Reset sequence snippet:



Main sequence snippet:



```

# UVM_INFO verilog_src/uvm-1.1d/src/base/uvm_objectection.svh(1267) @ 70310: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
# UVM_INFO SPI_score_board.sv(55) @ 70310: uvm_test_top.env.sb [report phase in score board] correct_count = 17470, error_count = 0
#
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO :17478
# UVM_WARNING : 0
# UVM_ERROR : 0
# UVM_FATAL : 0
# ** Report counts by id
# [Questa UVM] 2
# [RUNIST] 1
# [TEST_DONE] 1
# [report phase in score board] 1
# [run phase in score board] 17470
# [run phase in test] 3
# ** Note: $finish : E:/Programms/QuestaSim/win64/./verilog_src/uvm-1.1d/src/base/uvm_root.svh(430)
# Time: 70310 ns Iteration: 61 Instance: /top
# 1
# Break in Task uvm_pkg/uvm_root::run_test at E:/Programms/QuestaSim/win64/./verilog_src/uvm-1.1d/src/base/uvm_root.svh line 430

```

PART 2: RAM

Verification Plan:

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
RAM_1	Reset Functionality: When rst_n is low, all outputs (dout, tc_valid) and internal registers (Rd_Addr, Wr_Addr) must be reset to 0	ram_reset_sequence: Asserts rst_n = 0, tc_valid = 0, din = 0	N/A (single reset state)	Scoreboard: reference_model() checks reset condition and sets all golden model values to 0 Assertion: Reset_Outputs properly verifies tc_valid == 0 and dout == 0 when rst_n
RAM_2	Write Address Command (din[9:8] = 2'b00): When tc_valid is high and command is 2'b00, the lower 8 bits din[7:0] should be stored in Wr_Addr register	ram_writeOnly_sequence: Generates write address commands with Wr_ONLY constraint enabled. Random address values in din[7:0] ram_readWrite_sequence: Mixed read/write operations	CMD_cp: Coverpoint for write_addr bin (2'b00) CMD_RX_VALID_cross: Cross coverage between write_addr command and tc_valid = 1	Scoreboard: reference_model() case statement captures din[7:0] into Wr_Addr_GM when command is 2'b00
RAM_3	Write Data Command (din[9:8] = 2'b01): When tc_valid is high and command is 2'b01, the lower 8 bits din[7:0] should be written to memory location MEM[Wr_Addr]	ram_writeOnly_sequence: Generates write data commands following write address commands ram_readWrite_sequence: Ensures proper write address → write data sequence with post_randomize() logic	CMD_cp: Coverpoint for write_data bin (2'b01) Wr_DATA_AFTER_Wr_ADDR: Transition coverage (2'b00 => 2'b01) CMD_RX_VALID_cross: Cross coverage between write_data and tc_valid = 1	Scoreboard: reference_model() writes din[7:0] to MEM_GM[Wr_Addr_GM] when command is 2'b01 Assertion: Write_Address_Followed_By_Write_Data verifies write address is eventually followed by write data
RAM_4	Read Address Command (din[9:8] = 2'b10): When tc_valid is high and command is 2'b10, the lower 8 bits din[7:0] should be stored in Rd_Addr register	ram_readOnly_sequence: Generates read address commands with Rd_ONLY constraint enabled ram_readWrite_sequence: Mixed operations with read sequences	CMD_cp: Coverpoint for read_addr bin (2'b10) CMD_RX_VALID_cross: Cross coverage between read_addr and tc_valid = 1	Scoreboard: reference_model() captures din[7:0] into Rd_Addr_GM when command is 2'b10
RAM_5	Read Data Command (din[9:8] = 2'b11): When tc_valid is high and command is 2'b11, data from memory location MEM[Rd_Addr] should be output on dout and tc_valid should be asserted high	ram_readOnly_sequence: Generates read data commands following read address commands ram_readWrite_sequence: Ensures proper read address → read data sequence	CMD_cp: Coverpoint for read_data bin (2'b11) Rd_DATA_AFTER_Rd_ADDR: Transition coverage (2'b10 => 2'b11) Rd_DATA_TX_VALID_cross: Cross coverage between read_data command and tc_valid = 1	Scoreboard: Compares DUT dout with MEM_GM[Rd_Addr_GM] and tc_valid with expected value 1 Assertion: TX_Valid_Rise_After_Read_Data verifies tc_valid rises after read data command Assertion: Read_Address_Followed_By_Read_Data ensures read address is followed by read data
RAM_6	TX_Valid Signal Control: tc_valid should only be high for one clock cycle after a read data command (2'b11), and must be low for all other commands	All sequences: Default behavior ensures tc_valid pulses only during read data commands	TX_VALID_cp: Coverpoint for both tc_valid = 0 and tc_valid = 1 states	Scoreboard: Checks tc_valid_GM = 0 by default, only set to 1 during read data command Assertion: TX_Valid_Deasserted_During_Input verifies tc_valid == 0 after write/read address commands Assertion:
RAM_7	RX_Valid Control: Commands should only be processed when tc_valid is high. When tc_valid is low, no state changes occur	All sequences use tc_valid_c constraint with distribution tc_valid dist {1:= 95, 0:= 5} to test both active and idle cycles	RX_VALID_cp: Coverpoint for both tc_valid = 0 and tc_valid = 1 states CMD_RX_VALID_cross: Validates all commands occur with tc_valid = 1	Scoreboard: reference_model() only processes commands when ram_seq_ref.tc_valid is true
RAM_8	Command Transitions: Verify all valid command sequences and transitions between different operations	ram_readWrite_sequence: Uses post_randomize() to create realistic command transitions: - WRITE_ADDR → WRITE_DATA - WRITE_DATA → READ_ADDR (60%) or WRITE_ADDR (40%) - READ_ADDR → READ_DATA - READ_DATA → WRITE_ADDR (40%) or READ_ADDR (60%)	CMD_TRANS_cp: Transition coverage for 8 different command pairs: - write_addr → write_data - write_addr → read_addr - write_data → write_addr - write_data → read_addr - read_addr → write_addr - read_addr → read_data - read_data → write_addr - read_data → read_addr	Scoreboard: Validates correct behavior across all command transitions through comprehensive sequence testing
RAM_9	Full Write-Read Sequence: Verify complete write followed by read operation at same address returns correct data	ram_readWrite_sequence: 20,000 iterations of mixed write/read operations ensure multiple full sequences are executed	FULL_SEQUENCE: Coverpoint for complete sequence (2'b00 => 2'b01 => 2'b10 => 2'b11)	Scoreboard: Golden model MEM_GM stores written data and compares with read data, ensuring data integrity across write-read cycles
RAM_10	Address Space Coverage: Verify all 256 memory locations (0x00 to 0xFF) can be accessed for both read and write operations	All sequences generate random 8-bit addresses via din[7:0] randomization with uniform distribution across full address range	Implicit coverage through random address generation (can be enhanced with explicit address coverpoint if needed)	Scoreboard: Golden memory model MEM_GM[255:0] tracks all 256 locations, ensuring any address can be written and read correctly
RAM_11	Data Integrity: Verify written data is correctly stored and retrieved without corruption	All sequences use random data patterns in din[7:0] for write operations, followed by read operations to verify data	Implicit through transaction coverage	Scoreboard: Byte-by-byte comparison between seq_item_sb.dout (DUT output) and dout_GM (expected value from golden model) Error/correct counters track data integrity: error_count and correct_count
RAM_12	Output Default Values: When not performing a read data operation, dout should maintain its previous value and tc_valid should be 0	All sequences with non-read-data commands test default output behavior	Covered by general command coverage	Scoreboard: Verifies tc_valid_GM = 0 for all non-read-data operations Assertion: TX_Valid_Deasserted_During_Input ensures proper default behavior

Code Coverage Report:

```

Branch Coverage:
  Enabled Coverage      Bins      Hits      Misses      Coverage
  -----
  Branches              9         8         1      88.88%

=====Branch Details=====

Branch Coverage for file RAM.v --

-----IF Branch-----
  13                      17354      Count coming in to IF
  13          1           715          if (~rst_n) begin
  19          1          16639          else begin
Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----
  23                      16639      Count coming in to IF
  23          1          15816          if (rx_valid) begin
                          823          All False Count
Branch totals: 2 hits of 2 branches = 100.00%

-----CASE Branch-----
  24                      15816      Count coming in to CASE
  25          1          4533          2'b00 : Wr_Addr <= din[7:0];           // Write Address
  26          1          3488          2'b01 : MEM[Wr_Addr] <= din[7:0];       // Write Data
  27          1          4394          2'b10 : Rd_Addr <= din[7:0];       // Read Address
  28          1          3401          2'b11 : begin                       // Read Data
                          ***0***          All False Count
Branch totals: 4 hits of 5 branches = 80.00%

```


Statement Coverage:				
Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Statements	11	11	0	100.00%

=====Statement Details=====

Statement Coverage for file RAM.v --

Line	Bins	Hits	Misses	Coverage	Code
1					module RAM (din,clk,rst_n,rx_valid,dout,tx_valid);
2					
3					input [9:0] din;
4					input clk, rst_n, rx_valid;
5					
6					output reg [7:0] dout;
7					output reg tx_valid;
8					
9					reg [7:0] MEM [255:0];
10					reg [7:0] Rd_Addr, Wr_Addr;
11					
12	1	17354			always @(posedge clk or negedge rst_n) begin
13					if (~rst_n) begin
14	1	715			dout <= 0;
15	1	715			tx_valid <= 0;
16	1	715			Rd_Addr <= 0;
17	1	715			Wr_Addr <= 0;
18					end
19					else begin
20					// Default: tx_valid is low unless READ_DATA command
21	1	16639			tx_valid <= 0;
22					
23					if (rx_valid) begin
24					case (din[9:8])
25	1	4533			2'b00 : Wr_Addr <= din[7:0]; // Write Address
26	1	3488			2'b01 : MEM[Wr_Addr] <= din[7:0]; // Write Data
27	1	4394			2'b10 : Rd_Addr <= din[7:0]; // Read Address
28					2'b11 : begin // Read Data
29	1	3401			dout <= MEM[Rd_Addr];
30	1	3401			tx_valid <= 1'b1;
31					end
32					endcase
33					end
34					end
35					end
36					
37					endmodule

Toggle Coverage:				
Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Toggles	76	76	0	100.00%

=====Toggle Details=====

Toggle Coverage for File RAM.v --

Line	Node	1H->0L	0L->1H	"Coverage"
3	din[9]	1	1	100.00
3	din[8]	1	1	100.00
3	din[7]	1	1	100.00
3	din[6]	1	1	100.00
3	din[5]	1	1	100.00
3	din[4]	1	1	100.00
3	din[3]	1	1	100.00
3	din[2]	1	1	100.00
3	din[1]	1	1	100.00
3	din[0]	1	1	100.00
4	rx_valid	1	1	100.00
4	rst_n	1	1	100.00
4	clk	1	1	100.00
6	dout[7]	1	1	100.00
6	dout[6]	1	1	100.00
6	dout[5]	1	1	100.00
6	dout[4]	1	1	100.00
6	dout[3]	1	1	100.00
6	dout[2]	1	1	100.00
6	dout[1]	1	1	100.00
6	dout[0]	1	1	100.00
7	tx_valid	1	1	100.00
10	Wr_Addr[7]	1	1	100.00
10	Wr_Addr[6]	1	1	100.00
10	Wr_Addr[5]	1	1	100.00
10	Wr_Addr[4]	1	1	100.00
10	Wr_Addr[3]	1	1	100.00
10	Wr_Addr[2]	1	1	100.00
10	Wr_Addr[1]	1	1	100.00
10	Wr_Addr[0]	1	1	100.00
10	Rd_Addr[7]	1	1	100.00
10	Rd_Addr[6]	1	1	100.00
10	Rd_Addr[5]	1	1	100.00
10	Rd_Addr[4]	1	1	100.00
10	Rd_Addr[3]	1	1	100.00
10	Rd_Addr[2]	1	1	100.00
10	Rd_Addr[1]	1	1	100.00
10	Rd_Addr[0]	1	1	100.00

Total Node Count = 38
Toggled Node Count = 38
Untoggled Node Count = 0

Toggle Coverage = 100.00% (76 of 76 bins)

Code Coverage Analysis				
Statements - by instance (/ram_top/DUT)	Statement	✓	✗	E
RAM.v	12 always @(posedge clk or negedge rst_n) begin	✓		
	14 dout <= 0;	✓		
	15 tx_valid <= 0;	✓		
	16 Rd_Addr <= 0;	✓		
	17 Wr_Addr <= 0;	✓		
	21 tx_valid <= 0;	✓		
	25 2'b00 : Wr_Addr <= din[7:0];	✓		
	26 2'b01 : MEM[Wr_Addr] <= din[7:0];	✓		
	27 2'b10 : Rd_Addr <= din[7:0];	✓		
	29 dout <= MEM[Rd_Addr];	✓		
	30 tx_valid <= 1'b1;	✓		

Code Coverage Analysis				
Branches - by instance (/ram_top/DUT)	Branch	✓	✗	E
RAM.v	13 if (~rst_n) begin	✓		
	19 else begin	✓		
	23 if (rx_valid) begin	✓		
	24 case (din[9:8])	✗		
	25 2'b00 : Wr_Addr <= din[7:0];	✓		
	26 2'b01 : MEM[Wr_Addr] <= din[7:0];	✓		
	27 2'b10 : Rd_Addr <= din[7:0];	✓		
	28 2'b11 : begin // Read Data	✓		

This because the default value
by I put it outside the case

Coverage Analysis				
Toggle	✓	✗	E	
sim:/ram_top/DUT	✓			clk
	✓			din
	✓			dout
	✓			Rd_Addr
	✓			rst_n
	✓			rx_valid
	✓			tx_valid
	✓			Wr_Addr

Functional Coverage report & Assertions:

Coverage					
Group/ops	1	na	na	100.00%	
Coverpoints/Crosses	0	na	na	na	
Covergroup Hits	24	24	0	100.00%	
Covergroup			Metric	Goal	Status
TYPE ram_coverage_collector_pkg/ram_coverage/ram0_cg					
covered/total hits:			100.00%	100	Covered
missing/total hits:			0	24	-
% Hit:			100.00%	100	-
Coverpoint CMD_cp			100.00%	100	Covered
covered/total hits:			0	4	-
missing/total hits:			0	4	-
% Hit:			100.00%	100	-
Coverpoint CMD_TRAM_cp			100.00%	100	Covered
covered/total hits:			0	8	-
missing/total hits:			0	8	-
% Hit:			100.00%	100	-
Coverpoint rx_valld_cp			100.00%	100	Covered
covered/total hits:			2	2	-
missing/total hits:			0	2	-
% Hit:			100.00%	100	-
Coverpoint tx_valld_cp			100.00%	100	Covered
covered/total hits:			2	2	-
missing/total hits:			0	2	-
% Hit:			100.00%	100	-
Coverpoint rx_data_after_wr_addr			100.00%	100	Covered
covered/total hits:			1	1	-
missing/total hits:			0	1	-
% Hit:			100.00%	100	-
Coverpoint rx_data_after_rd_addr			100.00%	100	Covered
covered/total hits:			1	1	-
missing/total hits:			0	1	-
% Hit:			100.00%	100	-
Coverpoint FULL_SEQUENCE			100.00%	100	Covered
covered/total hits:			1	1	-
missing/total hits:			0	1	-
% Hit:			100.00%	100	-
Cross CMD_RX_VALID_cross			100.00%	100	Covered
covered/total hits:			4	4	-
missing/total hits:			0	4	-
% Hit:			100.00%	100	-
Cross RD_DATA_TX_VALID_cross			100.00%	100	Covered
covered/total hits:			1	1	-
missing/total hits:			0	1	-
% Hit:			100.00%	100	-
Covergroup instance vram_coverage_collector_pkg/vram_coverage/ram0_cg			100.00%	100	Covered

missing/total bins:	0	1	-
% Hit:	100.00%	100	-
Coverpoint FULL_SEQUENCE	100.00%	100	Covered
covered/total bins:	1	1	-
missing/total bins:	0	1	-
% Hit:	100.00%	100	-
Cross CMD_RX_VALID_cross	100.00%	100	Covered
covered/total bins:	4	4	-
missing/total bins:	0	1	-
% Hit:	100.00%	100	-
Cross RD_DATA_TX_VALID_cross	100.00%	100	Covered
covered/total bins:	1	1	-
missing/total bins:	0	1	-
% Hit:	100.00%	100	-
Covergroup Instance Vram_coverage_collector_pgl3:vram_coverage:RAM_cg	100.00%	100	Covered
covered/total bins:	0	24	24
missing/total bins:	0	24	24
% Hit:	100.00%	100	-
Coverpoint CMD_cp	100.00%	100	Covered
covered/total bins:	0	4	-
missing/total bins:	0	4	-
% Hit:	100.00%	100	-
bin write_addr	6241	1	Covered
bin write_data	4847	1	Covered
bin read_addr	6149	1	Covered
bin read_data	4763	1	Covered
Coverpoint CMD_READ_cp	100.00%	100	Covered
covered/total bins:	8	8	-
missing/total bins:	0	8	-
% Hit:	100.00%	100	-
bin write_addr_to_write_data	4586	1	Covered
bin write_addr_to_read_addr	1	1	Covered
bin write_data_to_write_addr	2024	1	Covered
bin write_data_to_read_addr	2562	1	Covered
bin read_addr_to_write_addr	1	1	Covered
bin read_addr_to_read_addr	4508	1	Covered
bin read_data_to_write_addr	2562	1	Covered
bin read_data_to_read_addr	1946	1	Covered
Coverpoint RX_VALID_cp	100.00%	100	Covered
covered/total bins:	2	2	-
missing/total bins:	0	2	-
% Hit:	100.00%	100	-
bin rx_valid_0	1869	1	Covered
bin rx_valid_1	20931	1	Covered
Coverpoint TX_VALID_cp	100.00%	100	Covered
covered/total bins:	2	2	-
missing/total bins:	0	2	-
% Hit:	100.00%	100	-
bin tx_valid_0	19778	1	Covered
bin tx_valid_1	4452	1	Covered
Coverpoint IM_DATA_AFTER_MR_ADDR	100.00%	100	Covered
covered/total bins:	1	1	-
missing/total bins:	0	1	-
% Hit:	100.00%	100	-
bin wr_data_after_wr_addr	4506	1	Covered

covered/total bins:	24	24	-
missing/total bins:	0	24	-
% Hit:	100.00%	100	-
Coveragepoint CMD_cp	100.00%	100	-
covered/total bins:	4	4	-
missing/total bins:	0	4	-
% Hit:	100.00%	100	-
bin write_addr	1	1	-
bin write_data	4847	1	-
bin read_addr	6149	1	-
bin read_data	4763	1	-
Coveragepoint CMD_TRANS_cp	100.00%	100	-
covered/total bins:	8	8	-
missing/total bins:	0	8	-
% Hit:	100.00%	100	-
bin write_addr_to_write_data	4586	1	-
bin write_addr_to_read_addr	1	1	-
bin write_data_to_write_addr	2024	1	-
bin write_data_to_read_addr	2562	1	-
bin read_addr_to_write_addr	1	1	-
bin read_addr_to_read_data	4508	1	-
bin read_data_to_write_addr	2562	1	-
bin read_data_to_read_addr	1946	1	-
Coveragepoint RX_VALID_cp	100.00%	100	-
covered/total bins:	2	2	-
missing/total bins:	0	2	-
% Hit:	100.00%	100	-
bin rx_valid_0	1069	1	-
bin rx_valid_1	20931	1	-
Coveragepoint TX_VALID_cp	100.00%	100	-
covered/total bins:	2	2	-
missing/total bins:	0	2	-
% Hit:	100.00%	100	-
bin tx_valid_0	17378	1	-
bin tx_valid_1	4422	1	-
Coveragepoint WR_DATA_AFTER_WR_ADDR	100.00%	100	-
covered/total bins:	1	1	-
missing/total bins:	0	1	-
% Hit:	100.00%	100	-
bin wr_data_after_wr_addr	4586	1	-
Coveragepoint RD_DATA_AFTER_RD_ADDR	100.00%	100	-
covered/total bins:	1	1	-
missing/total bins:	0	1	-
% Hit:	100.00%	100	-
bin rd_data_after_rd_addr	4586	1	-
Coveragepoint FULL_SEQUENCE	100.00%	100	-
covered/total bins:	1	1	-
missing/total bins:	0	1	-
% Hit:	100.00%	100	-
bin full_seq	1912	1	-
Cross CMD_RX_VALID_cross	100.00%	100	-
covered/total bins:	4	4	-
missing/total bins:	0	4	-
% Hit:	100.00%	100	-

Coverpoint RD_DATA_AFTER_RD_ADDR	100.00%	100	-	Covered
covered/total bins:		1	-	Covered
missing/total bins:		0	1	-
% Hit:	100.00%	100	-	
bin rd_data_after_rd_addr	4590	1	-	Covered
Coverpoint FULL_SEQUENCE	100.00%	100	-	Covered
covered/total bins:		1	1	-
missing/total bins:		0	1	-
% Hit:	100.00%	100	-	
bin full_seq	1912	1	-	Covered
Cross CMD_RX_VALID_cross	100.00%	100	-	Covered
covered/total bins:		4	4	-
missing/total bins:		0	1	-
% Hit:	100.00%	100	-	
Auto, Default and User Defined Bins:				
bin <read_data,rx_valid_1>	4523	1	-	Covered
bin <write_data,rx_valid_1>	4685	1	-	Covered
bin <read_addr,rx_valid_1>	5870	1	-	Covered
bin <write_addr,rx_valid_1>	5933	1	-	Covered
Illegal and Ignore Bins:				
ignore_bin rx_not_validd	1069	-	-	Occurred
Cross RD_DATA_TX_VALID_cross	100.00%	100	-	Covered
covered/total bins:		1	1	-
missing/total bins:		0	1	-
% Hit:	100.00%	100	-	
Auto, Default and User Defined Bins:				
bin <read_data,tx_valid_1>	4422	1	-	Covered
Illegal and Ignore Bins:				
ignore_bin tx_not_validd	17578	-	-	Occurred
ignore_bin not_read_data	17237	-	-	Occurred
TOTAL COVERAGE: 100.00%	COVERGROUP TYPES: 1			

DIRECTIVE COVERAGE:						
Name	Design Unit	Design UnitType	Lang	File(Line)	Hits	Status
/ram_top/DUT/RAM_DUT_SVA/cover__Read_Address_Followed_By_Read_Data	ram_SVA	Verilog	SVA	ram_sva.sv(75)	5554	Covered
/ram_top/DUT/RAM_DUT_SVA/cover__Write_Address_Followed_By_Write_Data	ram_SVA	Verilog	SVA	ram_sva.sv(64)	5631	Covered
/ram_top/DUT/RAM_DUT_SVA/cover__TX_Valid_Fall_After_Rise	ram_SVA	Verilog	SVA	ram_sva.sv(53)	4014	Covered
/ram_top/DUT/RAM_DUT_SVA/cover__TX_Valid_Rise_After_Read_Data	ram_SVA	Verilog	SVA	ram_sva.sv(42)	4341	Covered
/ram_top/DUT/RAM_DUT_SVA/cover__TX_Valid_Deasserted_During_Input	ram_SVA	Verilog	SVA	ram_sva.sv(31)	15751	Covered
/ram_top/DUT/RAM_DUT_SVA/cover__Reset_Outputs	ram_SVA	Verilog	SVA	ram_sva.sv(19)	446	Covered

TOTAL DIRECTIVE COVERAGE: 100.00% COVERS: 6

Total Coverage By Instance (filtered view): 100.00%

Coveragegroup	Metric	Goal	Bins	Status
TYPE /ram_coverage_collector_pkg/ram_coverage/RAM_cg				
covered/total bins:	100.00%	100	-	Covered
missing/total bins:	0	24	-	
% Hit:	100.00%	100	-	
Coverpoint QW_cp	100.00%	100	-	Covered
covered/total bins:	100.00%	100	-	
missing/total bins:	0	4	-	
% Hit:	100.00%	100	-	
Coverpoint QW_TRANS_cp	100.00%	100	-	Covered
covered/total bins:	100.00%	100	-	
missing/total bins:	0	8	-	
% Hit:	100.00%	100	-	
Coverpoint RX_VALID_cp	100.00%	100	-	Covered
covered/total bins:	100.00%	100	-	
missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
Coverpoint TX_VALID_cp	100.00%	100	-	Covered
covered/total bins:	100.00%	100	-	
missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
Coverpoint WR_DATA_AFTER_WR_ADDR	100.00%	100	-	Covered
covered/total bins:	100.00%	100	-	
missing/total bins:	0	1	-	
% Hit:	100.00%	100	-	
Coverpoint RD_DATA_AFTER_RD_ADDR	100.00%	100	-	Covered
covered/total bins:	100.00%	100	-	
missing/total bins:	0	1	-	
% Hit:	100.00%	100	-	
Coverpoint FULL_SEQUENCE	100.00%	100	-	Covered
covered/total bins:	100.00%	100	-	
missing/total bins:	0	1	-	
% Hit:	100.00%	100	-	
Cross QW_RX_VALID_cross	100.00%	100	-	Covered
covered/total bins:	100.00%	100	-	
missing/total bins:	0	4	-	
% Hit:	100.00%	100	-	

Cover Directives														
Name	Language	Enabled	Log	Count	Alt.east	Limit	Weight	Cmplt %	Cmplt graph	Included	Memory	Peak Memory	Peak Memory Time	Cumulative Threads
/ram_top/DUT/RAM_DUT_SVA/cover_Read_Address_Followed_By_Read_Data	SVA	✓	Off	5554	1 Unli...	1	100%		✓	0	0	0 ns	0	
/ram_top/DUT/RAM_DUT_SVA/cover_Write_Address_Followed_By_Write_Data	SVA	✓	Off	5631	1 Unli...	1	100%		✓	0	0	0 ns	0	
/ram_top/DUT/RAM_DUT_SVA/cover_TX_Valid_Fail_After_Rise	SVA	✓	Off	4014	1 Unli...	1	100%		✓	0	0	0 ns	0	
/ram_top/DUT/RAM_DUT_SVA/cover_TX_Valid_Rise_After_Read_Data	SVA	✓	Off	4341	1 Unli...	1	100%		✓	0	0	0 ns	0	
/ram_top/DUT/RAM_DUT_SVA/cover_TX_Valid_Deasserted_During_Inout	SVA	✓	Off	15751	1 Unli...	1	100%		✓	0	0	0 ns	0	
/ram_top/DUT/RAM_DUT_SVA/cover_Reset_Outputs	SVA	✓	Off	446	1 Unli...	1	100%		✓	0	0	0 ns	0	

Assertion Coverage:			
Assertions	6	6	0 100.00%
Name	File(Line)	Failure Count	Pass Count

/ram_top/DUT/RAM_DUT_SVA/Reset_Outputs_label	ram_sva.sv(16)	0	1
/ram_top/DUT/RAM_DUT_SVA/TX_Valid_Deasserted_label	ram_sva.sv(27)	0	1
/ram_top/DUT/RAM_DUT_SVA/TX_Valid_Rise_label	ram_sva.sv(39)	0	1
/ram_top/DUT/RAM_DUT_SVA/TX_Valid_Fall_label	ram_sva.sv(50)	0	1
/ram_top/DUT/RAM_DUT_SVA/Write_Addr_Wr_Data_label	ram_sva.sv(61)	0	1
/ram_top/DUT/RAM_DUT_SVA/Read_Addr_Rd_Data_label	ram_sva.sv(72)	0	1
=====			
=== Instance: /ram_sequence_pkg			
=== Design Unit: work.ram_sequence_pkg			
=====			

Assertion Coverage:			
Assertions	3	3	0 100.00%
Name	File(Line)	Failure Count	Pass Count

/ram_sequence_pkg/ram_writeOnly_sequence/body/#ublk#33711751#41/immed_50	ram_sequence.sv(50)	0	1
/ram_sequence_pkg/ram_readOnly_sequence/body/#ublk#33711751#78/immed_87	ram_sequence.sv(87)	0	1
/ram_sequence_pkg/ram_readWrite_sequence/body/#ublk#33711751#109/immed_118	ram_sequence.sv(118)	0	1
ASSTION RESULTS:			

Name	File(Line)	Failure Count	Pass Count

/ram_top/DUT/RAM_DUT_SVA/Reset_Outputs_label	ram_sva.sv(16)	0	1
/ram_top/DUT/RAM_DUT_SVA/TX_Valid_Deasserted_label	ram_sva.sv(27)	0	1
/ram_top/DUT/RAM_DUT_SVA/TX_Valid_Rise_label	ram_sva.sv(39)	0	1
/ram_top/DUT/RAM_DUT_SVA/TX_Valid_Fall_label	ram_sva.sv(50)	0	1
/ram_top/DUT/RAM_DUT_SVA/Write_Addr_Wr_Data_label	ram_sva.sv(61)	0	1
/ram_top/DUT/RAM_DUT_SVA/Read_Addr_Rd_Data_label	ram_sva.sv(72)	0	1
/ram_sequence_pkg/ram_writeOnly_sequence/body/#ublk#33711751#41/immed_50	ram_sequence.sv(50)	0	1
/ram_sequence_pkg/ram_readOnly_sequence/body/#ublk#33711751#78/immed_87	ram_sequence.sv(87)	0	1
/ram_sequence_pkg/ram_readWrite_sequence/body/#ublk#33711751#109/immed_118	ram_sequence.sv(118)	0	1
Total Coverage By Instance (filtered view): 100.00%			

▲ /ram_sequence_pkg/ram_writeOnly_sequence/body/#ublk#33711751#41/immed_50	Immediate	SVA	on	0	1	-	...	-	-	off	assert (randomize(...))	✓
▲ /ram_sequence_pkg/ram_readOnly_sequence/body/#ublk#33711751#78/immed_87	Immediate	SVA	on	0	1	-	...	-	-	off	assert (randomize(...))	✓
▲ /ram_sequence_pkg/ram_readWrite_sequence/body/#ublk#33711751#109/immed_118	Immediate	SVA	on	0	1	-	...	-	-	off	assert (randomize(...))	✓
▲ /ram_top/DUT/RAM_DUT_SVA/Reset_Outputs_label	Concurrent	SVA	on	0	1	-	...	0...	0 ns	0 off	assert(@(posedge clk) (~rst_n) ->{~tx_valid& dout==0})	✓
▲ /ram_top/DUT/RAM_DUT_SVA/TX_Valid_Deasserted_label	Concurrent	SVA	on	0	1	-	...	0...	0 ns	0 off	assert(@(posedge clk) disable iff (~rst_n) (((~tx_valid& din[9:8]) in...	✓
▲ /ram_top/DUT/RAM_DUT_SVA/TX_Valid_Rise_label	Concurrent	SVA	on	0	1	-	...	0...	0 ns	0 off	assert(@(posedge clk) disable iff (~rst_n) (((~tx_valid& din[9:8]) in...	✓
▲ /ram_top/DUT/RAM_DUT_SVA/TX_Valid_Fall_label	Concurrent	SVA	on	0	1	-	...	0...	0 ns	0 off	assert(@(posedge clk) disable iff (~rst_n) (((~tx_valid& din[9:8]) in...	✓
▲ /ram_top/DUT/RAM_DUT_SVA/Write_Addr_Wr_Data_label	Concurrent	SVA	on	0	1	-	...	0...	0 ns	0 off	assert(@(posedge clk) disable iff (~rst_n) (((~tx_valid& din[9:8]) in...	✓
▲ /ram_top/DUT/RAM_DUT_SVA/Read_Addr_Rd_Data_label	Concurrent	SVA	on	0	1	-	...	0...	0 ns	0 off	assert(@(posedge clk) disable iff (~rst_n) (((~tx_valid& din[9:8]) in...	✓

Feature	Assertion
Reset ensures tx_valid and dout are low	@(posedge clk) (!rst_n -> (tx_valid == 0 && dout == 0));
tx_valid is deasserted during input operations	@(posedge clk) disable iff(!rst_n) (rx_valid && (din[9:8] inside {2'b00, 2'b01, 2'b10})) -> (tx_valid == 0);
tx_valid rises after read data command	@(posedge clk) disable iff(!rst_n) (rx_valid && din[9:8] == 2'b11) -> ##1 (tx_valid == 1);
tx_valid falls after rising	@(posedge clk) disable iff(!rst_n) (\$rose(tx_valid)) > ##[0:1] (!tx_valid);
Write address is followed by write data	@(posedge clk) disable iff(!rst_n) (rx_valid && din[9:8] == 2'b00) -> ##[1:\$] (rx_valid && din[9:8] == 2'b01);
Read address is followed by read data	@(posedge clk) disable iff(!rst_n) (rx_valid && din[9:8] == 2'b10) -> ##[1:\$] (rx_valid && din[9:8] == 2'b11);

Bugs:

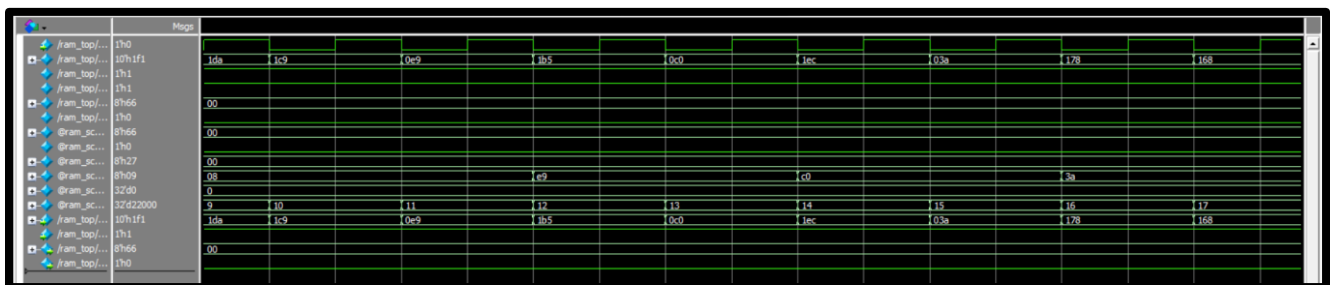
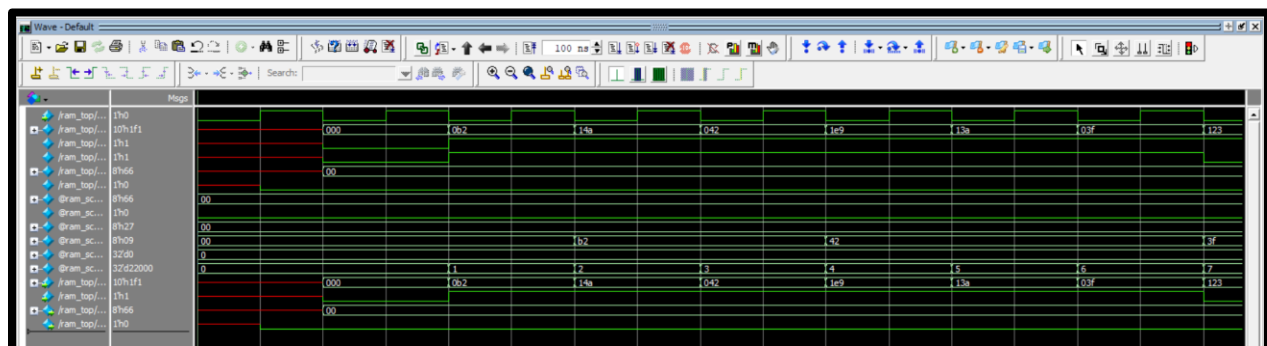
Bug 1 fix:

```
// Default: tx_valid is low unless READ_DATA command  
tx_valid <= 0;
```

Bug 2 fix:

```
2'b11 : begin                                // Read Data  
    dout <= MEM[Rd_Addr];  
    tx_valid <= 1'b1;  
end
```

Wave Form:



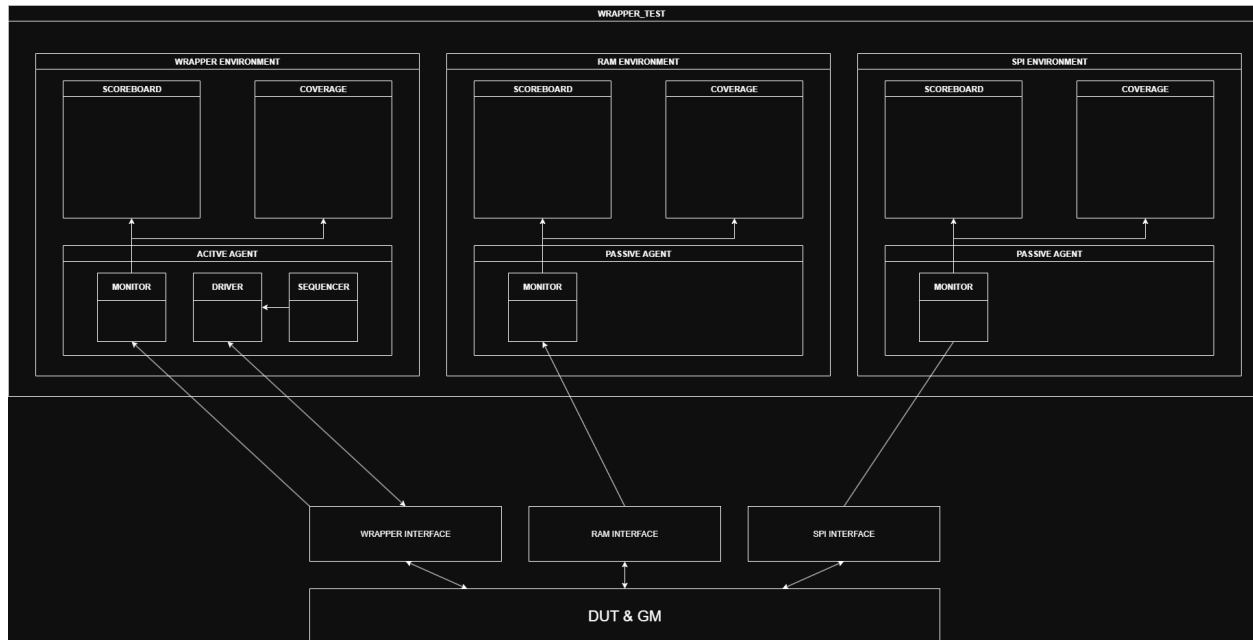
```
Transcript  
** Report counts by severity  
# UVM_INFO : 14  
# UVM_WARNING : 0  
# UVM_ERROR : 0  
# UVM_FATAL : 0  
** Report counts by id  
# [Questa UVM] 2  
# [RNTST] 1  
# [TESTI_DONE] 1  
# [report_phase] 2  
# [run_phase] 8  
# ** Note: $finish : C:/questasim64_2021.1/win64/./verilog_src/uvm-1.1d/src/base/uvm_root.svh(430)  
# Time: 44002 ns Iteration: 61 Instance: /ram_top  
# 1  
# Break in Task uvm_pkg/uvm_root::run_test at C:/questasim64_2021.1/win64/./verilog_src/uvm-1.1d/src/base/uvm_root.svh line 430
```

PART 3: WRAPPER

Verification Plan

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
WRAP_1	When reset is asserted, all outputs (MISO, rx_valid, rx_data) must remain inactive	Directed at start of simulation (rst_n=0)	–	ASSERT_RESET_INACTIVE assertion checks reset behavior;
WRAP_2	Data should be written correctly from SPI to RAM through the Wrapper	Randomized write sequences (write_only_sequence)	–	Golden model (scoreboard) compares written data; timing/order verified by testbench
WRAP_3	Data should be read correctly from RAM through Wrapper to SPI	Randomized read sequences (read_only_sequence)	–	Golden model compares read data with expected; ASSERT_MISO_STABLE ensures MISO changes only during READ
WRAP_4	Write followed by read from the same address should return identical data (end-to-end integrity)	Randomized write-read combined sequence (write_read_sequence)	–	Golden model compares readback vs. written data to verify integrity

UVM TEST



Code Snippets

Driver

We couldn't do the `ss_n` constraint from the sequence item class so achieved a similar effect from the driver `run_phase`

```
task run_phase(uvm_phase phase);
    super.run_phase(phase);

    forever begin
        // Get new transaction
        seq_item_port.get_next_item(stim_seq_item);

        // Connect interface signals
        WRAPPER_vif.rst_n = stim_seq_item.rst_n;
        WRAPPER_vif.MOSI_bits = stim_seq_item.MOSI_bits;

        // Determine SS_n period (13 or 23 cycles)
```

```

period_cycles = (stim_seq_item.MOSI_bits[10:8] == 3'b111)
? 23 : 13;

// Start of SPI frame
WRAPPER_vif.SS_n = 0;
SSn_cycle_count = 0;
//@(negedge WRAPPER_vif.clk);
// ---- Drive 11 MOSI bits (MSB first) ----
for (int i = 0; i < 11; i++) begin
    @(negedge WRAPPER_vif.clk);
    WRAPPER_vif.MOSI = stim_seq_item.MOSI_bits[10 - i];
    SSn_cycle_count++;
end

// ---- For Read Data, wait extra 8 cycles ----
if (stim_seq_item.MOSI_bits[10:8] == 3'b111) begin
    for (int j = 0; j < 8; j++) begin
        @(negedge WRAPPER_vif.clk);
        SSn_cycle_count++;
    end
end

// ---- Fill remaining cycles to complete the full period
----

extra_idle = period_cycles - SSn_cycle_count;
if (extra_idle > 0) begin
    repeat (extra_idle) begin
        @(negedge WRAPPER_vif.clk);
        SSn_cycle_count++;
    end
end

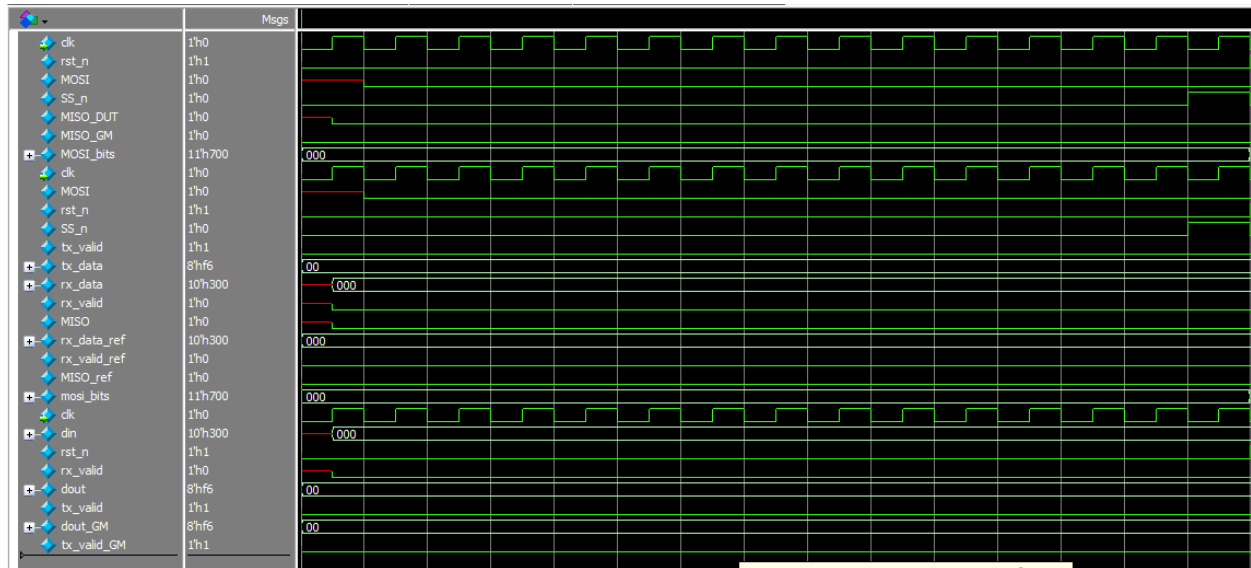
// ---- SS_n high for one cycle ----
@(negedge WRAPPER_vif.clk);
WRAPPER_vif.SS_n = 1;
@(negedge WRAPPER_vif.clk);
WRAPPER_vif.SS_n = 0;

// Prepare for next operation
stim_seq_item.op = stim_seq_item.next_op;

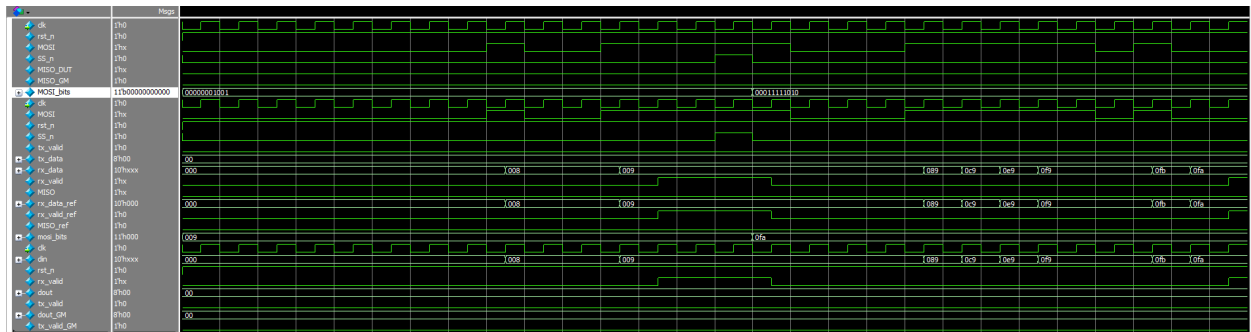
```

Wave Forms

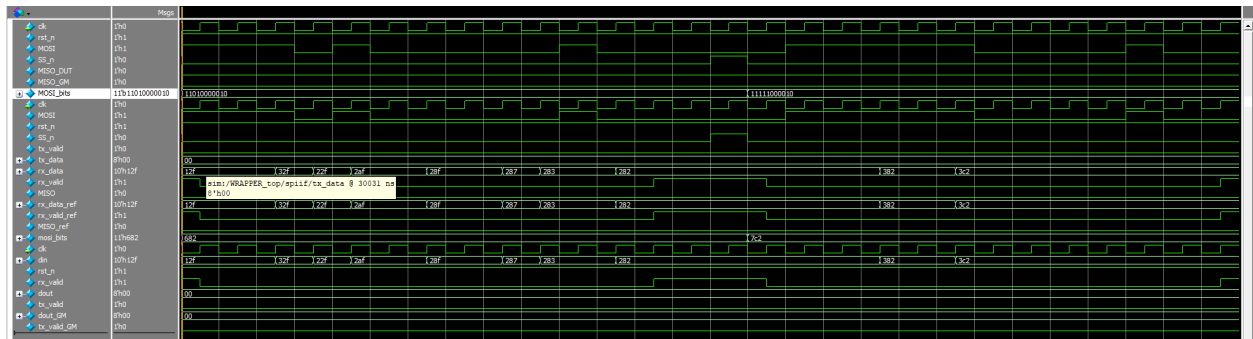
Reset



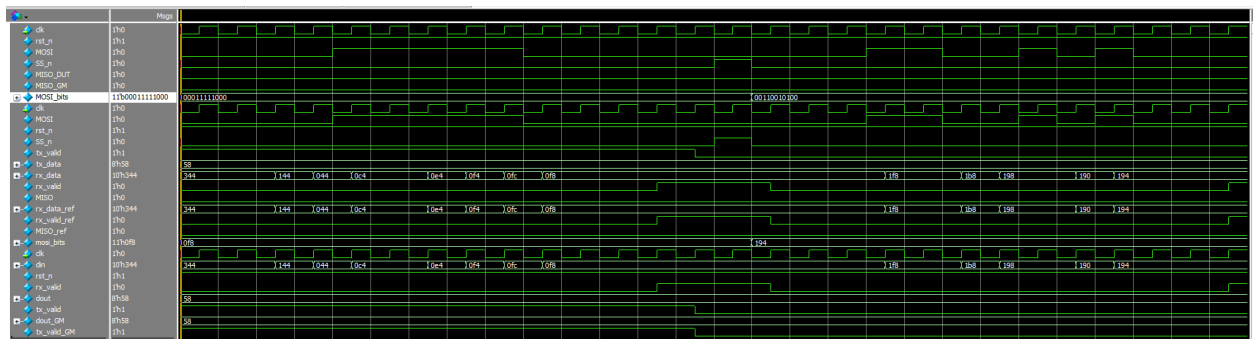
Write only sequence



Read only sequence



Write Read sequence



Assertions

Feature	Assertion
Outputs remain inactive during reset	<pre>`@(posedge clk) !rst_n ->##1 (MISO == 1'b0 && rx_valid == 1'b0 && rx_data == '0);</pre>
MISO remains stable when not in READ operation	<pre>`@(posedge clk) disable iff (!rst_n) (! (cs == 3'b100)) ->##1 \$stable(MISO);</pre>